

DNS Security 1

Cloud Security

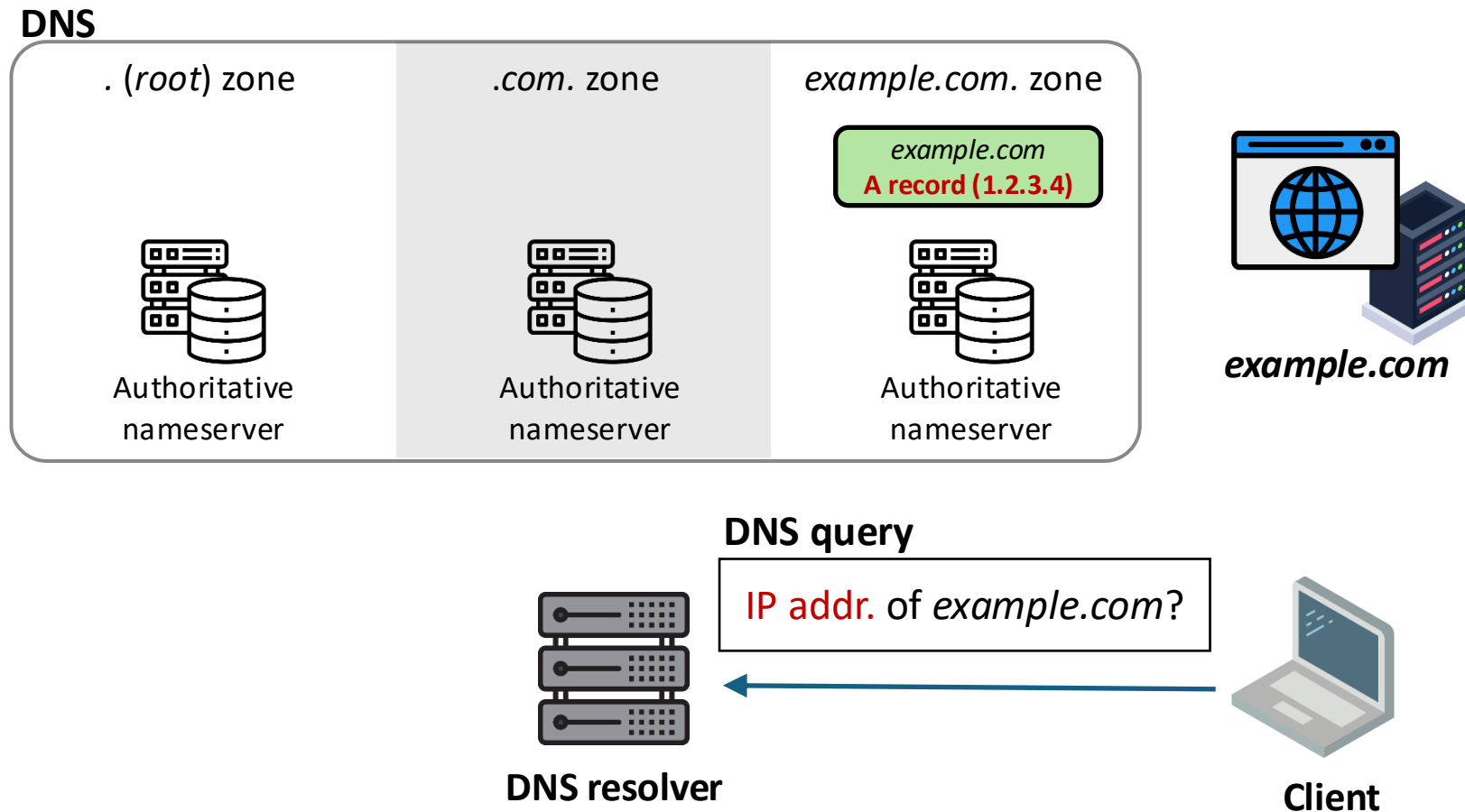
CSE4059

Revisit: Domain Name System (DNS)

- A distributed database that stores records that map domain names to values
 - `www.hanyang.ac.kr` → `166.104.177.20`
- The address lookup is the one that runs before essentially every connection
- DNS has a hierarchical structure
 - Each zone can create names only under the part of the tree they control
 - Each level knows only who is responsible for the level below it

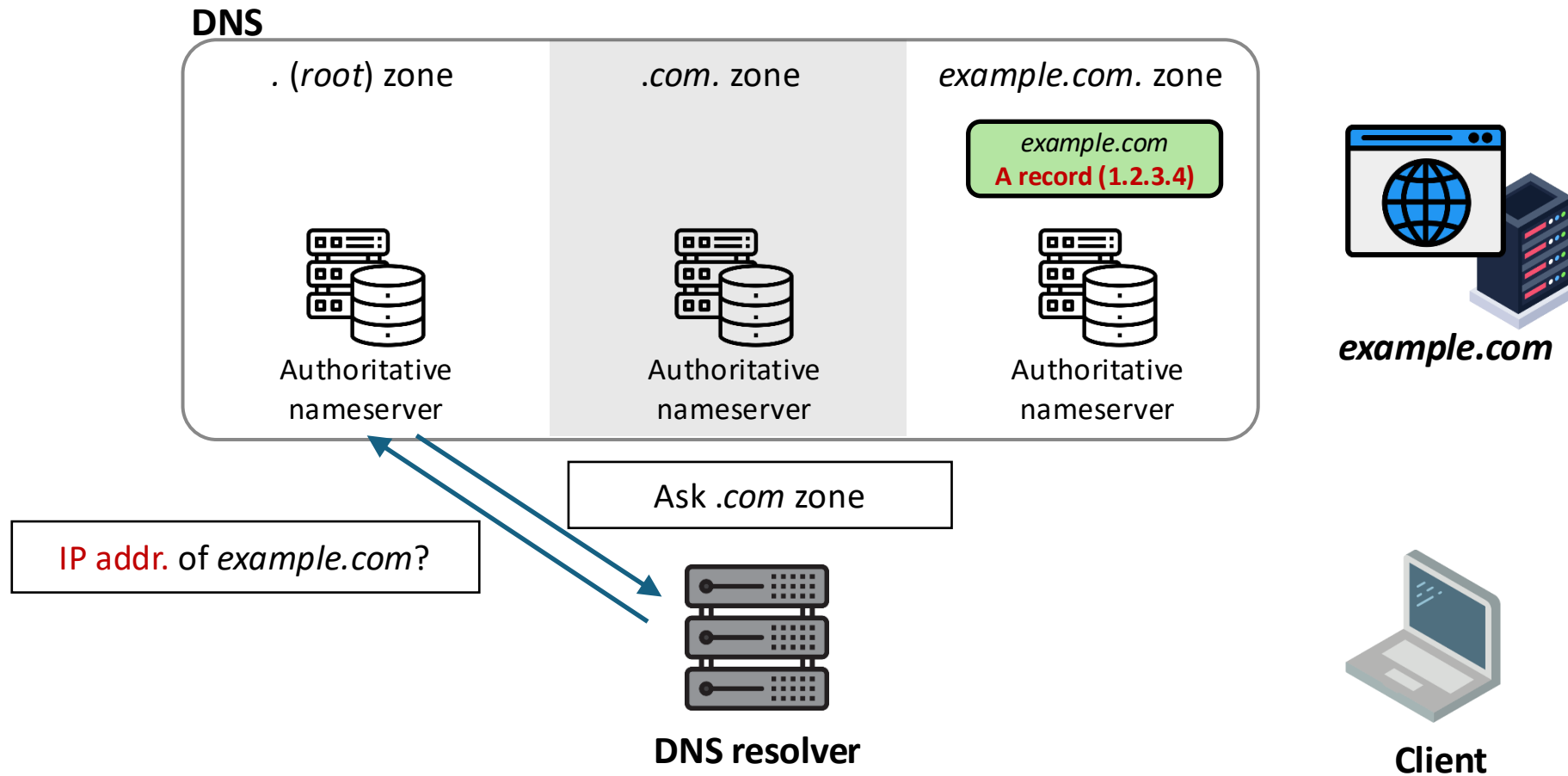
Revisit: DNS Resolution

Example – DNS Resolution



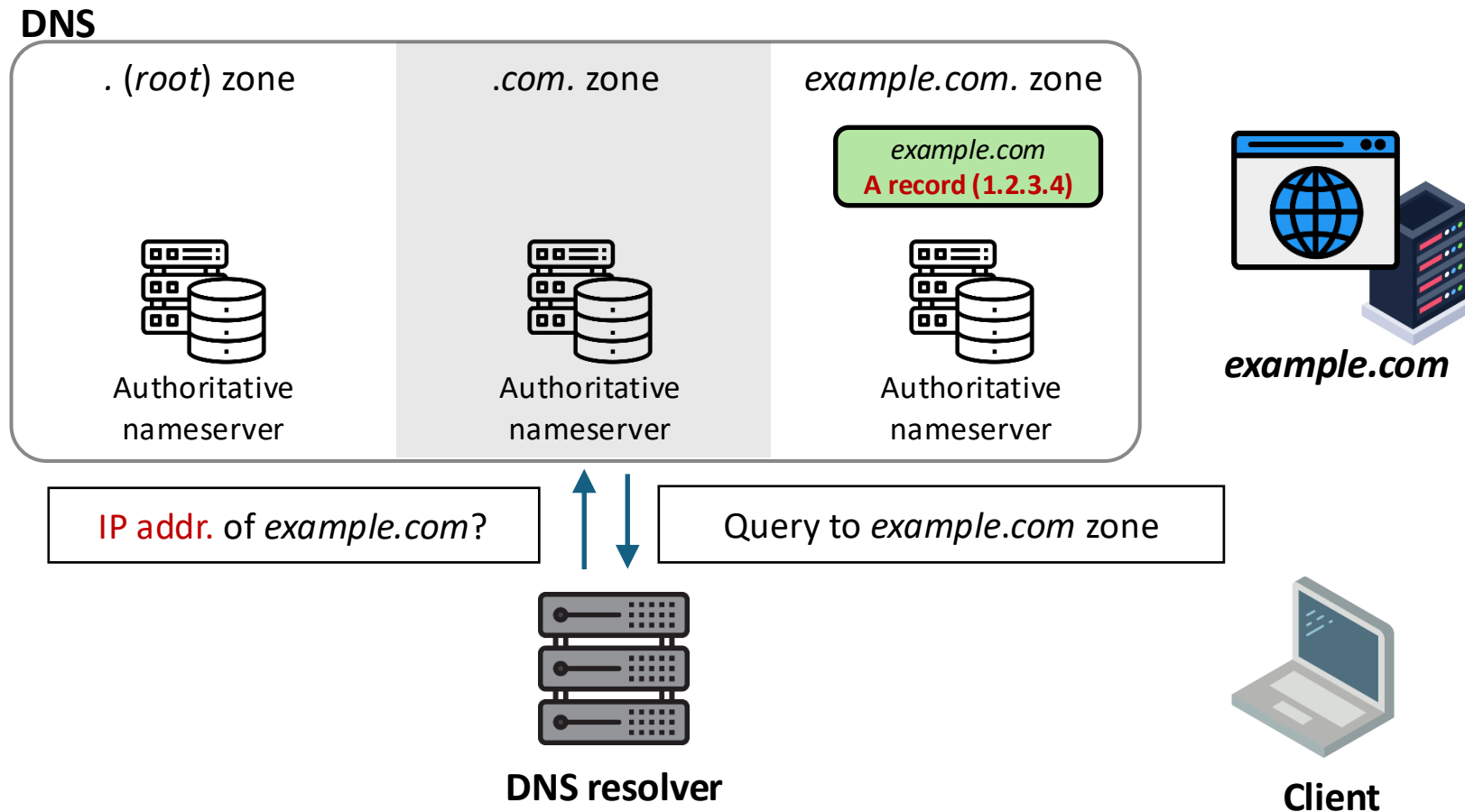
Revisit: DNS Resolution

Example – DNS Resolution



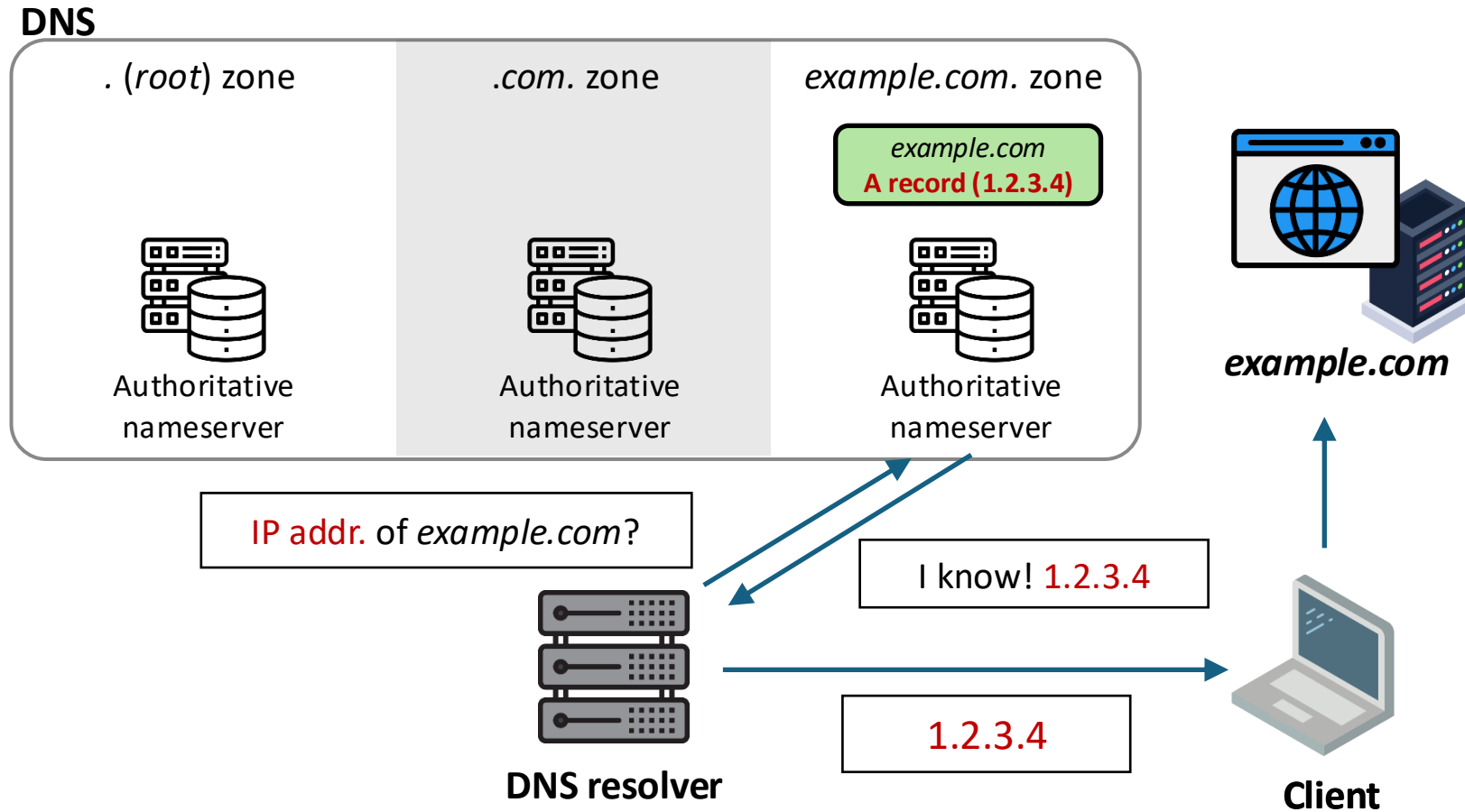
Revisit: DNS Resolution

Example – DNS Resolution



Revisit: DNS Resolution

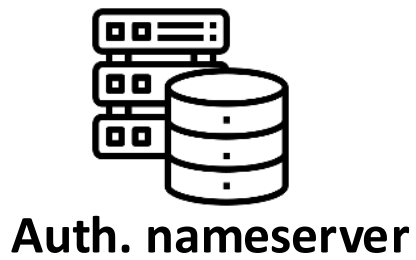
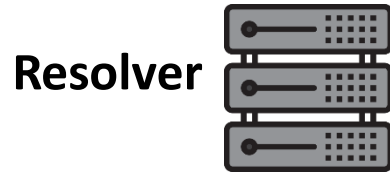
Example – DNS Resolution



Matching a DNS Response

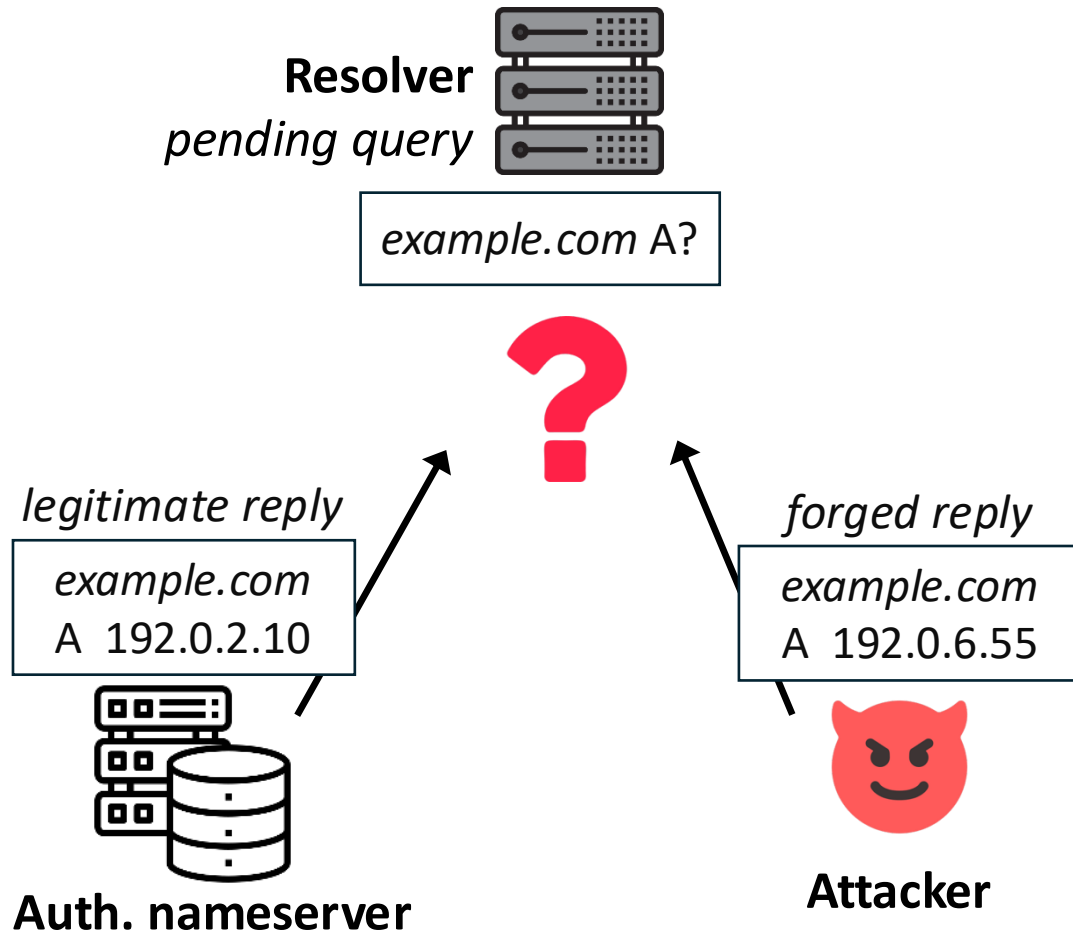
- The resolver asks an authoritative nameserver for an address
 - It must decide which response belongs to its query
- Example
 - Query: Resolver 192.0.2.53:53017 → Auth. nameserver 192.0.2.80:53
 - Question: *example.com* / A (IPv4 address) / Transaction ID: 0x4A21
 - Transaction ID? An identifier used to match the response to this query
 - Expected response: 192.0.2.80:53 → 192.0.2.53:53017
 - with the same question and Transaction ID
- Matching does not authenticate the DNS data

DNS Response Spoofing



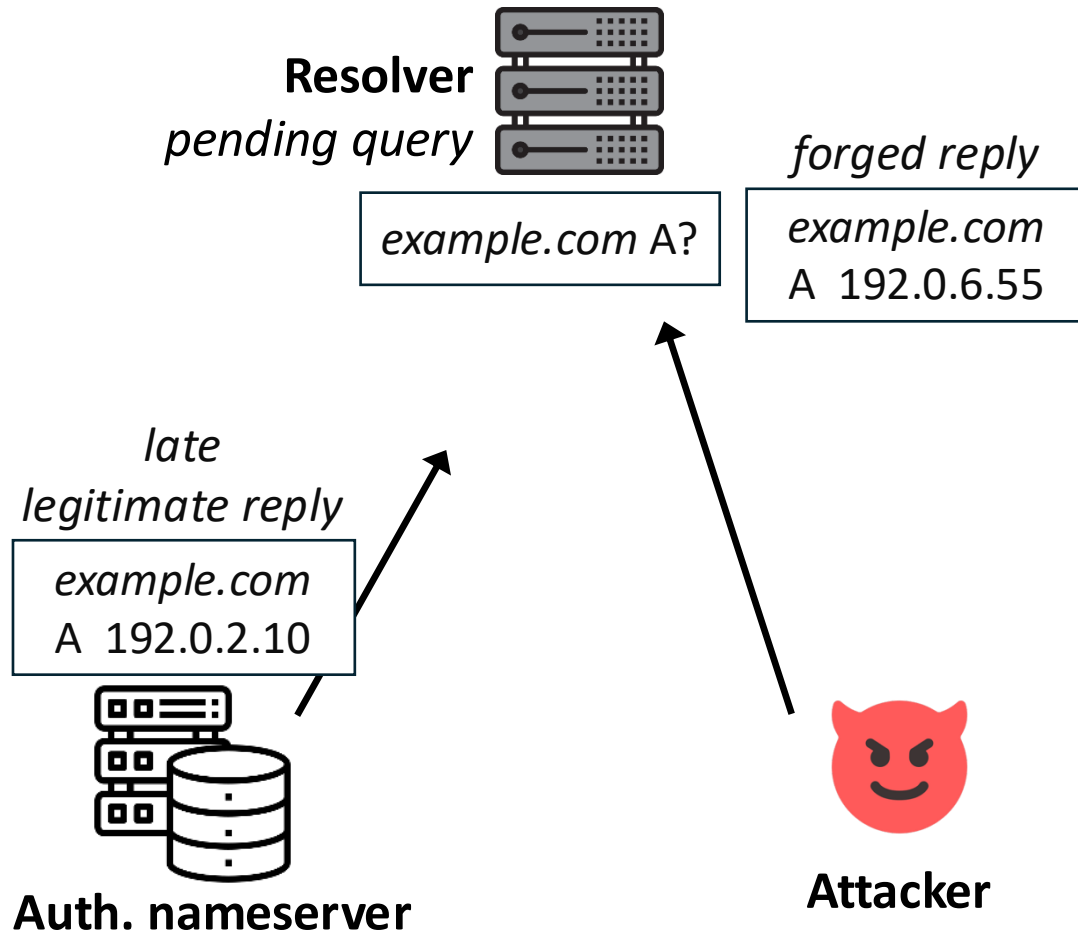
- DNS response spoofing inserts a forged reply that appears to answer a legitimate query
 - The attacker claims to be the expected responder and provides false record data
 - The resolver must accept the forged data for the attack to affect its answer
- Forging a packet and having that packet accepted are separate steps

Response Race: Pending Query



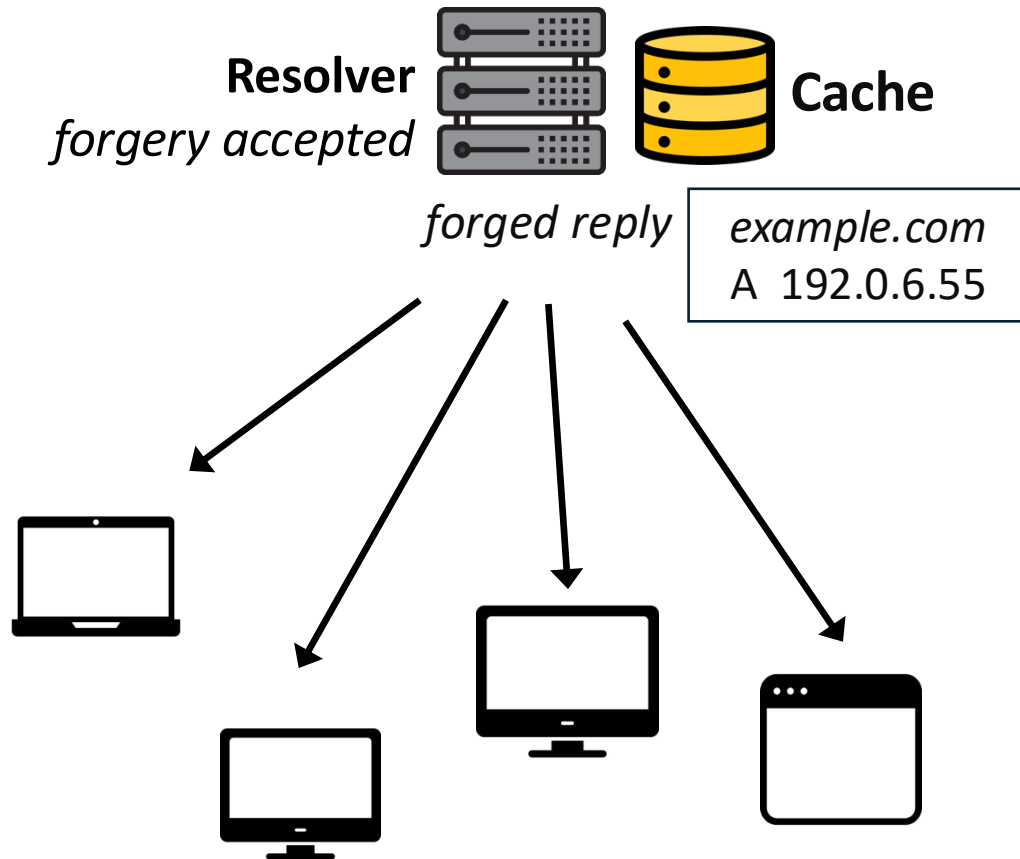
- Assumptions
 - Empty answer cache
 - One pending UDP query
 - No cryptographic authentication of DNS data
- Resolver
 - Query: *example.com* / A record
- The auth. nameserver sends
 - 192.0.2.10
- The attacker sends
 - 192.0.6.55
- Same query, two competing responses

Response Race: Forged Answer Accepted



- The forged reply arrives while the query is pending and passes the resolver's checks
 - Transaction ID, question, addresses, and ports match the outstanding exchange
- The resolver returns the forged address 192.0.6.55
 - Also, caches the accepted record
- The legitimate reply (192.0.2.10) arrives after this exchange has completed
 - But the late reply does not automatically replace the completed result

DNS Cache Poisoning



- DNS cache poisoning places poisoned DNS data in a cache that later requests reuse
 - Spoofing is one possible insertion method
 - Poisoning describes the stored result
- Clients will receive the forged DNS answer
- The impact follows cache reuse, not a new successful spoofing race for every client
- TTL limits cached reuse
 - But it does not prove that the cached data is authentic

Response Race: Rejected Forgery

- Possible reasons for the forged response is rejected
- The outstanding query *example.com* / A record / ID 0x4A21 / port 53017
 - A forged response with ID 0x4A22 does not match that query
 - A forged response sent to a different port does not match the expected exchange
 - A forged response that arrives after the query has completed is too late
- Rejecting these forged responses does not prove that every matching response is authentic

Different Records, Different Impact

- A record maps a particular name to an IPv4 address
 - A forged A record can redirect that name to an attacker-controlled address
- NS record identifies the authoritative nameservers for a zone
 - A forged NS record can redirect subsequent DNS resolution for that zone to attacker-controlled nameservers
 - The potential impact can be broader than poisoning a single A record
- The impact depends on which record is forged, whether the resolver accepts it, and how long it is cached

Different Sources of False DNS Data

- Forged response
 - An attacker injects data into a legitimate lookup
- Compromised resolver
 - The resolver itself returns altered answers
- Compromised domain administration
 - Unauthorized changes become authoritative data
- Client configuration change
 - The client sends queries to an attacker-selected resolver

On-Path and Off-Path Attackers

- An ***on-path*** attacker can observe the DNS exchange
 - It can learn the transaction ID, question, IP addresses, and ports
- An ***off-path*** attacker cannot directly observe that exchange
 - It must guess unknown matching values
- The same defense can have different value against these two positions

Transaction ID and Port Randomization

- Against off-path attackers, unpredictable transaction IDs and source ports make forged responses harder to match
 - The attacker cannot observe the query and must guess these values
- Randomization therefore raises the cost of off-path spoofing
 - Transaction ID
 - Port number
- On-path attackers can still observe DNS messages and learn these values

Other Operational Defenses

- Use trusted, designated resolvers
 - Prevent clients from using attacker-controlled resolvers
- Protect resolver configuration and administration
 - Keep resolver software updated
 - Restrict who can change resolver settings
- Protect authoritative DNS records
 - Prevent unauthorized changes to A, NS, and other records
- These defenses reduce attack opportunities

DNS Tunneling

- DNS tunneling carries non-DNS application data inside DNS messages
 - An endpoint encodes data in query names or response records
- Valid DNS exchanges carry *unauthorized* application data
 - The messages can be valid DNS even when the communication violates network policy
 - This is misuse of a permitted channel
 - cf. earlier attacks (e.g., cache poisoning) changed DNS answers



Endpoint
Software



Resolver



Auth.
Nameserver

DNS Tunneling – Participants and Assumptions

- Participants
 - **Endpoint:** software that can generate DNS queries and read responses
 - **Resolver:** permitted DNS service that resolves DNS queries
 - **Attacker authority (Auth. nameserver):** the entity that controls a domain name (*example.com*) and can receive its queries
- Assumptions
 - The separate data exchange is unauthorized
 - The setup uses valid DNS exchanges and does not require a false address



Endpoint
Software



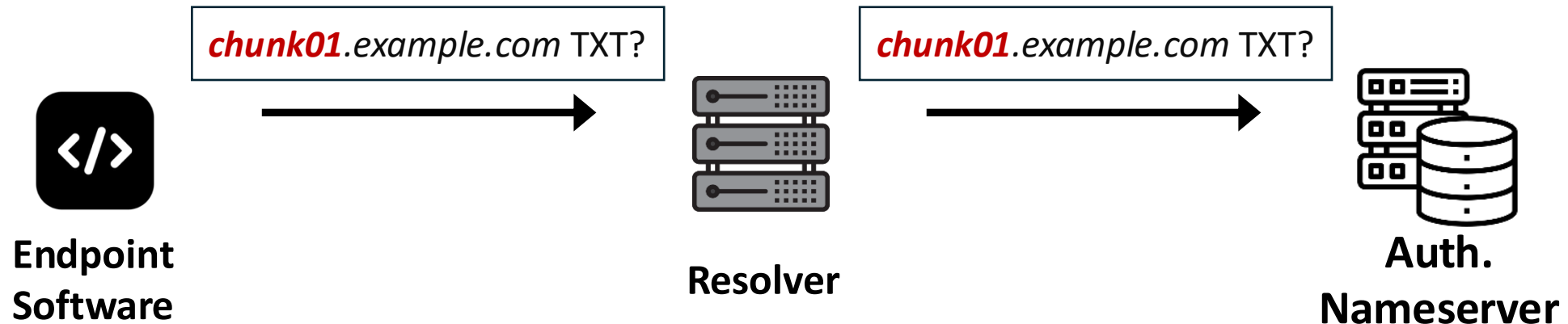
Resolver



Auth.
Nameserver

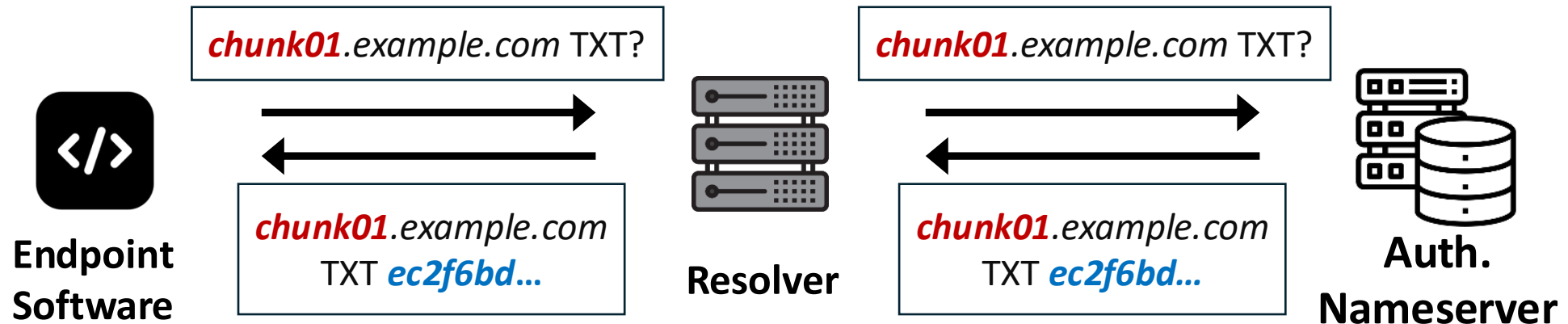
DNS Tunneling – Data in Query Name

- Example query: *chunk01.example.com* / TXT record
 - *chunk01* carries encoded application data
 - The encoding makes the data suitable for inclusion in a query name
- The endpoint sends the query to its resolver
 - An DNS lookup reaches the attacker's nameserver
- The nameserver reads the query name and interprets it as data



DNS Tunneling – Data in DNS Response

- Example response for *chunk01.example.com*: TXT “*ec2f6bd...*”
 - A TXT record carries *text data* as DNS record content
- The resolver returns the response to the requesting endpoint
- The endpoint interprets “*ec2f6bd...*” using the tunnel’s separate application rules



DNS Tunneling – Uses in Attacks

- Data exfiltration
 - Unauthorized transfer of data out of a (compromised) system
 - DNS query names can carry encoded pieces of data toward an attacker-controlled domain
- Command and Control (C2)
 - Malware on a compromised endpoint periodically communicates with an attacker (via DNS)
 - DNS responses can carry attack commands
 - ex) instruct infected hosts to launch a DDoS attack against a target
- DNS tunneling requires a compromised or cooperating endpoint
 - DNS tunneling is typically used after compromise
 - DNS itself does not cause the initial compromise

DNS Tunneling – Effect of DNS Caching

- A repeated query can be answered from cache
 - It may not reach the authoritative nameserver again
- DNS tunnels therefore often use different data-bearing names (e.g., subdomains)
 - Ex) *chunk01.example.com* and *chunk02.example.com*
- Each unique name can trigger a new DNS lookup
 - Allowing different pieces of data to reach the authoritative nameserver

DNS Tunneling – Traffic Characteristics

- DNS tunneling may generate many unique subdomains
- Query names may be long or contain encoded-looking data
- TXT records may be used to carry larger amounts of data
- These characteristics arise from using DNS as a data channel
 - Unique names can carry different pieces of data
 - DNS query/response can carry encoded information

DNS Tunneling – Detection Signals

- Traffic-based signals
 - Sustained or unusual DNS activity
 - Many unique or long query names
 - Encoded-looking subdomains
- Context-based signals
 - Unexpected process generating DNS queries
 - Unknown or unusual destination domains
- No single signal proves DNS tunneling
 - Legitimate services can show similar patterns
 - Detection should combine traffic patterns and context

Example: Interpreting Suspicious Queries

Common observation

- Many changing subdomains under one domain

Case A

- Approved software
 - Known service purpose
 - Traffic matches expected behavior
- Likely legitimate: the query pattern alone is not sufficient to classify it as tunneling

Case B

- Unexpected process
 - Unknown or unauthorized destination
 - Repeated data-like DNS exchanges
- Potentially malicious: the combined evidence warrants further investigation

- Same DNS pattern, different context

Defense with Port-based Filtering

- A common DNS security control is port-based filtering
 - Allow DNS traffic only to resolvers over port 53
- A port-based rule can identify DNS traffic, but not what the query name contains
- Tunnel data can still be embedded in DNS queries sent through a resolver
- Effective DNS tunneling detection requires combining the traffic- and context-based signals

What Problems Still Remain?

- Previous defenses can reduce DNS spoofing and tunneling risks
 - Transaction ID / source-port randomization
 - Resolver and endpoint policies
- But the traditional DNS still has fundamental limitations
 - Privacy: DNS queries and responses can be observed on the path
 - Data authentication: DNS data has no cryptographic proof of origin or integrity

Summary

- Traditional DNS does not protect DNS data
- DNS spoofing can inject false answers, and cache poisoning can make them persist
- DNS tunneling misuses valid DNS traffic as a data channel